<table>
<tr><td colspan="2">Nombre: Emmanuel Carrillo Loera

Edgar Daniel del Ángel Hernández

Fatima Alejandra Cavazos Serrato

Equipo: 3</td><td>Matrícula: 2067988

2094613

2099488</td></tr>
<tr><td>Nombre del curso: Componentes de Lógica de Negocios</td><td colspan="2">Nombre del profesor: Miguel Ángel Gómez Marroquín</td></tr>
<tr><td>Modulo Netacad: 3</td><td colspan="2">Actividad: Patrón de lógica de negocio de actividad fundamental 3</td></tr>
<tr><td colspan="3">Fecha: 13/10/2026</td></tr>
<tr><td colspan="3">Bibliografía (Formato APA)

Archivo “Actividad_Unidad_3_CLN_ArquitecturaAlumnos.docx” proporcionado en la plataforma TEAMS

Archivo “03_Sazón_Urbano.docx” proporcionado en la plataforma TEAMS</td></tr>
</table>

Diseño de arquitectura del sistema

Preparando el caso

El proceso principal es la gestión del pedido, desde que el cliente ordena por cualquier canal hasta que el pedido se entrega, se cancela o la entrega no se completa.

Casos de uso

ID	Caso de uso	Actor	Objetivo	Información de entrada	Reglas relacionadas	Resultado esperado
CU-01	Registrar pedido	Mesero (mesa) · Caja (teléfono, WhatsApp, domicilio)	Capturar el pedido completo y enviarlo a cocina	Canal, productos, cantidades, modificaciones, mesa o datos del cliente (teléfono y dirección si es domicilio), responsable	RN-01, RN-14 · NEC-11, NEC-12	Pedido confirmado, con responsable y canal, enviado completo a las estaciones. Rechazo: domicilio < $150 o sin dirección/teléfono
CU-02	Aplicar promoción	Caja	Aplicar un descuento válido al pedido	Pedido, promoción seleccionada, fecha y hora	RN-04, RN-05, RN-09	Pedido con una sola promoción vigente y total recalculado. Rechazo: fuera de vigencia o segunda promoción
CU-03	Cancelar pedido	Mesero o Caja (solicitan) · Encargado (autoriza)	Anular el pedido controlando la merma	Pedido, motivo, estado de las estaciones, autorización	RN-02, RN-03, RN-10 · NEC-04	Pedido cancelado con motivo y responsable; merma si ya había preparación. Rechazo: requiere autorización del Encargado
CU-04	Gestionar producto agotado	**4a** Cocina (marca) · **4b** Mesero	Evitar pedidos incumplibles sin	4a: producto o insumo · 4b: pedido y línea	RN-06, RN-12 · NEC-03	4a: producto no disponible y aviso inmediato · 4b: pedido sin

		(quita línea)	cancelarlos completos			esa línea, total recalculado, cancelación parcial registrada
CU-05	Asignar repartidor y registrar entrega	Encargado (asigna) · Repartidor (registra)	Entregar el pedido a domicilio con tiempos registrados	Pedido listo, repartidor disponible, hora de salida y entrega, motivo si falla	RN-11, RN-07 · NEC-06	Pedido entregado con tiempos, o entrega no completada con motivo. Rechazo: entrega fallida

Elegir el patrón de lógica de negocio

Patrón principal: Domain Model

Combinación delimitada: Transaction Script solo en CU-04a (marcar agotado)

Table Module: descartado en los 5 casos; queda como candidato para los reportes (US-13 y US-17), que no forman parte de la selección

Tabla comparativa

Criterio del caso	Transaction script	Table Module	Domain Model	Evidencia U2
Complejidad de reglas	Con 6 reglas en 5 casos, cada script repetiría validaciones	Reglas atadas a datos, no a comportamiento	Cada regla vive en el objeto que la protege	RN-01, 02, 03, 04, 05, 12 · US-03, 07, 09
Cambios esperados	Un cambio de promoción obliga a editar varios scripts	Cambio localizado por tabla, pero mezclado con SQL	El cambio queda en una sola clase	US-18 (vigencia configurable), entrevista (2x1 de martes aplicado “de memoria”), RN-07, RN-08, RN-15
Interacción entre conceptos	Pedido, línea, estación, promoción, merma y entrega se cruzan dentro de un flujo largo	Cada módulo ve su tabla; la interacción queda fuera	Los objetos colaboran con sus reglas	RN-03 (cancelación + estación + merma + autorización), US-05 (recálculo del total)

Estados e invariantes	Hay que releer el estado en cada script	Sin modelo de ciclo de vida	Estado y transiciones protegidos	BPMN (¿Empezó preparación?, ¿Entrega exitosa?), RN-02, RN-04, RN-11
Adicional 1: misma regla en varios canales	Riesgo de duplicarla por canal	Posible, atada a la tabla	Una vez, sin importar el canal	BRD (mesa, teléfono/WhatsApp, domicilio) · US-01, US-02
Adicional 2: consultas y reportes	Posible	Ideal	Pesado para solo leer	US-13, US-17 · NEC-08, NEC-09

Ventajas, riesgos y consecuencias

Patrón	Ventajas	Riesgos	Consecuencia en el caso
Transaction Script	Rápido y claro para operaciones lineales (CU-04a)	Duplica lógica al crecer y mezcla reglas	Si se usara para el pedido, RN-02/03/12 se repetirían en cada flujo y cualquier cambio tocaría varios scripts
Table Module	Muy bueno para reportes y datos agregados	No modela el estado por estación ni las invariantes	Las reglas terminarían en servicios o en SQL, acopladas a las tablas
Domain Model	Reglas, estado y datos juntos; una regla, un lugar	Más clases y más estructura inicial; riesgo de sobreingeniería	Costo inicial mayor, que mitigamos limitando los aggregates (`Pedido`, `Entrega`) y exigiendo Repository y Unit of Work (Etapa 5)

Decisión argumentada

Elegimos Domain Model como patrón principal porque el problema real de Sazón Urbano no es la cantidad de funciones, sino que sus reglas dependen del estado del pedido y deben cumplirse igual en todos los canales. Cancelar un pedido (RN-02 y RN-03) exige saber si alguna estación ya inició la preparación; esa respuesta determina si se cancela directo o con autorización del Encargado, y si se genera merma. Con Transaction Script esa condición se repetiría en cada punto de entrada (mesa, teléfono, WhatsApp, domicilio), y cualquier ajuste, como las promociones que la entrevista describe como aplicadas “de memoria”, obligaría a editar varios scripts. Con Table Module no existiría un lugar natural para el ciclo de vida del pedido, porque ese patrón organiza la lógica alrededor de tablas.

Domain Model ubica cada invariante en el objeto que la protege: una sola promoción por pedido (RN-04), monto mínimo de $150 en domicilio (RN-01) y registro obligatorio de hora de salida o motivo en la entrega (RN-11). Además, el pedido se reutiliza en cuatro de los cinco casos, lo que justifica centralizarlo en un aggregate.

Transaction Script se usa solo en CU-04a (marcar agotado), porque es una operación lineal sin regla compleja. Table Module queda fuera de los casos seleccionados; sería la opción natural para los reportes de discrepancias y ventas (US-13 y US-17), pero esos casos no forman parte de nuestra selección.

Más estructura inicial y más clases que con scripts, además de la obligación de definir repositorios y límites transaccionales. Lo acotamos con dos aggregates principales y evitando convertir cada tabla en una entidad.